

Relatorio - Mini E-commerce Distribuido

Visao geral

O projeto foi implementado como um mini e-commerce distribuido composto por quatro aplicacoes Spring Boot independentes: gateway, users, products e orders. O gateway roda na porta 8080 e funciona como ponto unico de entrada. Os demais servicos rodam nas portas 5001, 5002 e 5003, respectivamente. A persistencia foi simplificada com arquivos JSON locais para manter a execucao leve e sem dependencia obrigatoria de Docker ou banco externo.

Comunicacao entre microservicos

A comunicacao externa acontece sempre pelo API Gateway. O Gateway recebe requisicoes em rotas como `/users/**`, `/products/**` e `/orders/**`, identifica o servico responsavel e encaminha a chamada HTTP para a URL interna configurada por variaveis de ambiente. Durante esse encaminhamento, headers importantes, como `Authorization`, sao preservados para que os servicos internos consigam validar o JWT.

Existe tambem comunicacao direta entre servicos no fluxo de pedidos. Ao criar um pedido, o servico orders consulta o servico products por HTTP para confirmar que o produto informado existe antes de gravar o pedido. Dessa forma, o pedido nao depende de uma copia local do catalogo de produtos e usa o servico de produtos como fonte da informacao.

O Gateway possui heartbeat periodico para os servicos internos. A cada intervalo configurado, ele chama os endpoints `/health` de usuarios, produtos e pedidos. Depois de duas falhas consecutivas, o servico e marcado como indisponivel. Quando volta a responder, o Gateway registra a recuperacao e libera novamente o roteamento.

Replicacao e consistencia de produtos

O servico products usa duas replicas de armazenamento em arquivos JSON: uma primaria e uma secundaria. As leituras usam uma estrategia simples de round-robin entre as replicas, distribuindo as consultas entre os arquivos disponiveis. Ja as escritas seguem consistencia forte: a criacao de um produto so retorna sucesso depois que as duas replicas foram gravadas.

Essa escolha evita que o cliente receba confirmacao de um produto que existe em apenas uma replica. Se qualquer uma das gravacoes falhar, a operacao e tratada como erro e a criacao nao e confirmada. Em um sistema real, essa abordagem poderia ser substituida por replicacao assincrona, filas ou consenso, mas para este projeto ela deixa o comportamento previsivel e facil de testar.

Queda do servico de pedidos

Se o servico orders cair, o Gateway continua funcionando e as rotas de usuarios e produtos permanecem disponiveis. Apos o limite de falhas do heartbeat, o Gateway marca orders como indisponivel. Nesse estado, qualquer chamada para `/orders/**` recebe 503 Service Unavailable, sem tentar encaminhar a requisicao para um servico que ja foi detectado como fora do ar.

Quando o servico de pedidos e iniciado novamente e volta a responder ao `/health`, o Gateway registra a recuperacao e passa a encaminhar as chamadas de pedidos outra vez. Assim, a falha fica isolada no servico afetado e nao bloqueia todo o e-commerce.

Uso de JWT e bloqueio por perfil

O servico users gera JWT no login contendo dados como `userId`, `email`, `role` e `expiracao`. Esse token e enviado pelo cliente no header `Authorization: Bearer`. Os servicos que precisam de autenticao validam a assinatura do token usando a mesma chave configurada em `JWT_SECRET`.

Para a criacao de produtos, o servico products exige que o token seja valido e que o campo `role` seja `admin`. Um usuario comum, com `role=user`, consegue consultar produtos e criar pedidos, mas ao tentar enviar `POST /products` recebe erro de autorizacao. Isso separa operacoes administrativas das operacoes comuns do cliente.

Limitacoes em relacao a um e-commerce real

Esta implementacao atende ao objetivo academico de demonstrar microsservicos, gateway, JWT, replicacao simples e heartbeat, mas ainda possui limitacoes importantes. A persistencia em JSON nao substitui um banco transacional, nao ha controle real de concorrência para estoque, nao existe pagamento, carrinho, entrega, cancelamento ou historico detalhado. Tambem nao ha observabilidade completa com metricas, tracing distribuido ou painel de monitoramento.

Outro ponto e que o JWT usa uma chave de ambiente simples e nao ha rotacao de chaves, refresh token ou revogacao de sessoes. A replicacao de produtos ocorre dentro do proprio servico em arquivos locais, enquanto um ambiente de producao usaria replicas independentes, banco distribuido, mensageria ou outro mecanismo resiliente. Mesmo assim, a arquitetura criada demonstra os principais conceitos pedidos: separacao por servico, comunicacao HTTP, autorizacao por perfil, tolerancia parcial a falhas e documentacao de execucao.